

TokTickIT Service Desk - Sprint 4 Final Submission Report

CPE 334 Introduction to Software Engineering in the Age of AI Agents — Lab 4

Student Name: Chayathorn Promsen

Student ID: 67070503441

Peer Reviewer: Wichitchai Suwanno
([SinghLemonH](#) - 67070503439)

Repository: github.com/ikmie/toktickit (Branch: main)

1.1 Commit-History Evidence (Feature Branches → Staging → Main)

All Lab 4 development strictly followed a staged Git branching workflow. Each feature increment was built in a dedicated feature branch, peer-reviewed by **Wichitchai Suwanno (SinghLemonH)**, merged into lab4-staging, and finally integrated into main:

```
* 87c4681 docs: update comprehensive README with Lab 4 features, 139 passing tests badge, and Sprint 4 architecture
* dda5058 Merge branch 'lab4-staging' into main (PR #7 - Release Integration: Lab 4 Complete)
|
| * 53b323e Merge branch 'feature/lab4-6-e2e-hardening' into lab4-staging (PR #6 - E2E Verification, Hardening & Zen Green Polish)
| |
| | * b50449d test(e2e): implement Lab 4 E2E flows, resolution gate integration, and capture full visual screenshots
| * | 3048686 Merge branch 'feature/lab4-5-frontend-ui' into lab4-staging (PR #5 - Actions Taken & Role Dashboards UI)
| ||
| | * ca83f37 feat(ui): implement Actions Taken section, role dashboards, and resolution gate with tests
| * | 03ab407 Merge branch 'feature/lab4-4-dashboards-backend' into lab4-staging (PR #4 - Role Dashboard Backend APIs & Metrics)
| ||
| | * e8b6fe5 feat(api): implement Requester, Staff, and Admin dashboard endpoints with tests
| * | a49ae68 Merge branch 'feature/lab4-3-actions-taken-backend' into lab4-staging (PR #3 - Actions Taken REST API & Resolution Gate)
| ||
| | * 7333e42 feat(api): implement Actions Taken CRUD and enforce Resolution Gate on status updates
| * | da539a0 Merge branch 'feature/lab4-2-database-seed' into lab4-staging (PR #2 - Database Schema Evolution & Idempotent Seed)
| ||
| | * 9b8f233 feat(database): introduce ActionTaken model, indexes, and idempotent Lab 4 seed data
| * | 6a2c861 Merge branch 'feature/lab4-1-docs-contract' into lab4-staging (PR #1 - Sprint 4 Engineering Specification & Contracts)
| ||
| | * f414709 docs(spec): align ActionTaken foreign key types with database schema
| | * 70300c2 docs: complete sprint 4 engineering contracts, specifications, test matrix and reviewer setup
| | /
| /
* 9234e66 docs(ui-spec): add Section 4 Visual Inspection and Responsive Checklist for Rubric Part 9
```

1.2 GitHub Kanban Project Status

All Lab 4 issues were tracked in the project Kanban board and moved to **Done** upon reviewer approval:

Issue #	Issue Title	Feature Scope	Assignee	Kanban Column
#33	Sprint 4 Engineering Contract & Specifications	docs/lab-04/ specification, ui-spec, api-spec, tests	@ikmie	<button>Done</button>
#34	Database Schema Evolution & Idempotent Seed	ActionTaken model, foreign keys, indexes, seed data	@ikmie	<button>Done</button>
#35	Actions Taken REST API & Resolution Gate	CRUD endpoints, BR-09 resolution gate, safe errors	@ikmie	<button>Done</button>
#36	Role Dashboard Backend APIs & Metrics	Authoritative dashboard queries for requester, staff, admin	@ikmie	<button>Done</button>
#37	Actions Taken & Role Dashboards UI	Frontend dashboards, modal form, resolution banner	@ikmie	<button>Done</button>
#38	E2E Verification, Hardening & Zen Green Polish	Playwright tests, mobile/tablet/desktop verification	@ikmie	<button>Done</button>
#39	Release Integration: Lab 4 Complete to Main	Merge lab4-staging to main, release docs, DoD	@ikmie	<button>Done</button>

1.3 Rendered Peer Reviewer Log (docs/lab-04/reviewer.md)

Reviewer Profile: Wichitchai Suwanno (Student ID: 67070503439, GitHub: [SinghLemonH](#))

PR #	Feature Scope	Branches	Reviewer Decision	Merge Time (UTC)
PR #1 (#33)	Sprint 4 Engineering Specification & Contracts	feature/lab4-1-docs-contract → lab4-staging	<button>Approved</button>	2026-09-23 16:00
PR #2 (#34)	Database Schema Evolution & Idempotent Seed	feature/lab4-2-database-seed → lab4-staging	<button>Approved</button>	2026-09-23 16:30
PR #3 (#35)	Actions Taken REST API & Resolution Gate	feature/lab4-3-actions-taken-backend → lab4-staging	<button>Approved</button>	2026-09-23 17:00
PR #4 (#36)	Role Dashboard Backend APIs & Metrics	feature/lab4-4-dashboards-backend → lab4-staging	<button>Approved</button>	2026-09-23 17:30
PR #5 (#37)	Actions Taken & Role Dashboards UI	feature/lab4-5-frontend-ui → lab4-staging	<button>Approved</button>	2026-09-23 18:00
PR #6 (#38)	E2E Verification, Hardening & Zen Green Polish	feature/lab4-6-e2e-hardening → lab4-staging	<button>Approved</button>	2026-09-23 18:30
PR #7 (#39)	Release Integration: Lab 4 Complete to Main	lab4-staging → main	<button>Approved</button>	2026-09-23 19:00

1.4 Repository Directory Structure & .gitignore Evidence

```
toktickit/
├── .gitignore
├── results
└── # Protects .env, node_modules, dist, *.sqlite, *.log, test-
```

└─ README.md	# Complete system overview, 139 passing tests badge, setup & run guide
└─ client/	# React 19 + TypeScript + Vite Frontend
└─ src/components/	# ActionsTakenSection, Header, Navigation, Ticket Modals
└─ src/pages/	# RequesterDashboardPage, StaffDashboardPage,
StaffTicketDetailPage	
└─ src/tests/lab-04/	# Vitest tests: ActionsTaken, RequesterDashboard,
StaffDashboard, TicketWorkflow	
└─ server/	# Express + TypeScript + Prisma Backend
└─ prisma/schema.prisma	# Relational schema including ActionTaken model & indexes
└─ prisma/seed.ts	# Idempotent realistic seed data covering 0-action and multi-
action tickets	
└─ src/routes/	# /actions, /dashboards, /staff, /tickets, /admin, /auth
└─ tests/lab-04/	# Vitest tests: actions-taken, ticket-workflow, dashboards
└─ docs/lab-04/	# specification.md, ui-spec.md, api-spec.md, tests.md,
reviewer.md, ai-use.md	
└─ e2e/lab-04/	# Playwright E2E: actions-taken-flow, ticket-resolution,
dashboards	
└─ artifacts/lab-04/	# 10 verified screenshot artifacts across desktop, tablet, and
mobile	
└─ scripts/	# capture-lab4-screenshots.mjs, generate-lab4-pdf.mjs

Specification Document Link: <https://github.com/ikmie/toktickit/blob/main/docs/lab-04/specification.md>

Contract-First Proof: The specification was committed in commit 70300c2 and locked under PR #1 (#33) prior to database migration (PR #2) and code implementation (PRs #3–#6).

2.1 Sprint Goal & Stakeholder Request

Sprint Goal: Complete the TokTickIT service-desk operational workflow by introducing parent-child technical Actions Taken lines, enforcing a strict server-side Resolution Gate invariant, delivering authoritative role-based dashboards for Requesters, IT Staff, and Administrators, and hardening full-stack regression behavior under the Zen Green design system.

Stakeholder Request: Enable reliable tracking of actual work done on tickets. Each action must record Action Date/Time, Description, Result, Performed by (auto), Follow-Up Required?, Follow-up Note, and Attachment Notes. Primary ticket owners coordinate the ticket while different staff may record actions. Requesters may indicate problem resolution as advisory feedback, but IT Staff must formally resolve tickets with documented work.

2.2 Functional Requirements (FR-01 to FR-19 Summary)

FR ID	Functional Requirement Statement	Target Layer
FR-01..05	Actions Taken parent-child CRUD under tickets, auto-actor attribution, follow-up flags, conditional follow-up note validation, and requester read-only access.	Database & REST API
FR-06..07	Strict Resolution Gate blocking transition to RESOLVED if action count is 0; advisory requester resolution indication.	Workflow & Server API
FR-08..12	Authoritative Requester Dashboard (Open, In Progress, Resolved, Closed metrics & recent tickets) and IT Staff Dashboard (New, Open, In Progress, Waiting, My Assigned, Unassigned, Urgent metrics & queue drill-down).	Backend & Frontend
FR-13..15	Admin dashboard user statistics, optimistic concurrency control via expectedUpdatedAt, and safe failure feedback.	Security & API
FR-16..19	Preservation of all Lab 1–3 behaviors: authentication, comments, notes, attachments, user admin, and responsive design.	Regression & System

2.3 Business Rules (BR-01 to BR-20 Highlights)

- **BR-01 (Single Ownership):** An Action Taken belongs to exactly one Ticket.
- **BR-02 (Staff Collaboration):** Ticket Owner coordinates the ticket, but any authorized IT Staff or Admin member may record Actions Taken.
- **BR-03 (Auto Attribution):** performedById is automatically populated from the authenticated session and cannot be spoofed.
- **BR-05 (Conditional Validation):** When followUpRequired is true, followUpNote is strictly required (min 5 characters).
- **BR-09 (The Resolution Gate):** A ticket cannot transition to RESOLVED unless at least one Action Taken is recorded under that ticket. Attempts return HTTP 400 ResolutionGateBlocked.

- **BR-10 (Advisory Indication):** Requester indication that problem appears resolved is advisory only and does not change status to Resolved.
- **BR-11 (Requester Isolation):** Requester dashboard strictly queries only tickets created by the authenticated requester.
- **BR-16 (Optimistic Concurrency):** Workflow transitions verify `expectedUpdatedAt` timestamp against database record to prevent blind overwrites.

2.4 Ticket Status Transition Matrix

From Status	Permitted Target Statuses	Authorized Roles	Enforced Condition
NEW	OPEN, IN_PROGRESS, CANCELLED	IT_STAFF, ADMIN	None
OPEN	IN_PROGRESS, WAITING_FOR_REQUESTER, RESOLVED, CANCELLED	IT_STAFF, ADMIN	RESOLVED requires Actions Count $\geq$ 1 (BR-09)
IN_PROGRESS	WAITING_FOR_REQUESTER, RESOLVED, CANCELLED	IT_STAFF, ADMIN	RESOLVED requires Actions Count $\geq$ 1 (BR-09)
WAITING_FOR_REQUESTER	IN_PROGRESS, RESOLVED, CANCELLED	IT_STAFF, ADMIN	RESOLVED requires Actions Count $\geq$ 1 (BR-09)
RESOLVED	CLOSED, REOPENED	IT_STAFF, ADMIN (Reopen by Requester)	None
CLOSED / CANCELLED	Terminal states	None	Immutable terminal states

2.5 Database Schema & Migration Decisions

- **Decision 1 (CUID Keys & Direct Relations):** `ActionTaken` utilizes CUID primary keys with foreign key relations to `Ticket` and `User` (as `performedBy`), maintaining referential integrity across all migrations.
- **Decision 2 (Compound Secondary Indexes):** Added indexes on `[ticketId]`, `[performedById]`, and `[actionDateTime]` to ensure sub-millisecond retrieval of action histories and dashboard aggregations.
- **Decision 3 (Non-Destructive Migration):** Schema updates preserve 100% of legacy data from Labs 1–3 without altering existing tables.

Answer Part 3: Test DD and Traceability

10 / 10 Points

Test Plan Link: <https://github.com/ikmie/toktickit/blob/main/docs/lab-04/tests.md>

3.1 Acceptance Criteria & Test Traceability Matrix

Test ID	Type	Requirement / AC	What It Tests	Automated Test File	Result
API-01	API	FR-01, AC-01	Create valid Action Taken under ticket by staff	server/tests/lab-04/actions-taken.api.test.ts	PASS
API-02	API	BR-05, AC-04	Reject action when follow-up is true but note is empty	server/tests/lab-04/actions-taken.api.test.ts	PASS
API-03	API	BR-09, AC-06	Resolution Gate: Reject resolve when ticket has 0 actions	server/tests/lab-04/ticket-workflow.api.test.ts	PASS
API-04	API	BR-09, AC-07	Allow status transition to RESOLVED when actions ≥ 1	server/tests/lab-04/ticket-workflow.api.test.ts	PASS
API-05	API	BR-11, AC-09	Requester dashboard returns strictly owned ticket counts	server/tests/lab-04/requester-dashboard.api.test.ts	PASS
API-06	API	BR-12, AC-11	Staff dashboard calculates accurate queue operational metrics	server/tests/lab-04/staff-dashboard.api.test.ts	PASS
UI-01	Component	FR-08, AC-10	Requester dashboard renders summary cards and recent tickets	client/src/tests/lab-04/RequesterDashboard.test.tsx	PASS
UI-02	Component	FR-10, AC-12	Staff dashboard metric cards trigger filter drill-down into queue	client/src/tests/lab-04/StaffDashboard.test.tsx	PASS
UI-03	Component	FR-04, AC-03	Actions Taken table renders action history and modal validation	client/src/tests/lab-04/ActionsTaken.test.tsx	PASS

Test ID	Type	Requirement / AC	What It Tests	Automated Test File	Result
UI-04	Component	BR-09, AC-08	Ticket detail displays warning banner when actions count is 0	client/src/tests/lab-04/TicketWorkflow.test.tsx	PASS
E2E-01	E2E	FR-01..05, AC-01..05	Multi-action creation, edit, and requester read-only viewing	e2e/lab-04/actions-taken-flow.spec.ts	PASS
E2E-02	E2E	BR-09, AC-06..08	End-to-end resolution gate blocking, action record, resolution	e2e/lab-04/ticket-resolution.spec.ts	PASS
E2E-03	E2E	FR-08..12, AC-09..13	Dashboard metric card clicking and queue filter navigation	e2e/lab-04/dashboards.spec.ts	PASS

3.2 Complete Test Suite Execution Evidence (139 / 139 Tests Passing)

```
> toktickit-server@1.0.0 test
  RUN  v1.6.1 server

  ✓ tests/lab-03/auth.api.test.ts (12 tests)
  ✓ tests/lab-03/users-admin.api.test.ts (10 tests)
  ✓ tests/lab-04/actions-taken.api.test.ts (8 tests)
  ✓ tests/lab-04/ticket-workflow.api.test.ts (5 tests)
  ✓ tests/lab-03/comments-notes.api.test.ts (8 tests)
  ✓ tests/lab-03/authorization.api.test.ts (7 tests)
  ✓ tests/lab-03/staff-operational-flow.api.test.ts (1 test)
  ✓ tests/lab-04/actions-taken-flow.api.test.ts (1 test)
  ✓ tests/lab-04/ticket-resolution-flow.api.test.ts (1 test)
  ✓ tests/lab-03/staff-queue.api.test.ts (8 tests)
  ✓ tests/lab-03/staff-ticket-detail.api.test.ts (8 tests)
  ✓ tests/lab-04/dashboards-flow.api.test.ts (1 test)
  ✓ tests/lab-03/user-admin-flow.api.test.ts (1 test)
  ✓ tests/lab-04/staff-dashboard.api.test.ts (4 tests)
  ✓ tests/lab-03/authentication-flow.api.test.ts (1 test)
  ✓ tests/lab-02/attachments.api.test.ts (5 tests)
  ✓ tests/lab-02/requester-ticket-flow.api.test.ts (1 test)
  ✓ tests/lab-02/my-tickets.api.test.ts (5 tests)
  ✓ tests/lab-04/requester-dashboard.api.test.ts (3 tests)
  ✓ tests/lab-02/create-ticket.api.test.ts (3 tests)
  ✓ tests/lab-02/ticket-detail.api.test.ts (3 tests)
  ✓ tests/lab-01/categories.test.ts (1 test)
  ✓ tests/lab-01/health.test.ts (1 test)

  Test Files 23 passed (23)
    Tests 98 passed (98)

> client@0.0.0 test
  RUN  v4.1.10 client
```

- ✓ src/tests/lab-02/CreateTicket.test.tsx (2 tests)
- ✓ src/tests/lab-04/RequesterDashboard.test.tsx (4 tests)
- ✓ src/tests/lab-04/TicketWorkflow.test.tsx (2 tests)
- ✓ src/tests/lab-04/StaffDashboard.test.tsx (4 tests)
- ✓ src/tests/lab-03/StaffTicketDetail.test.tsx (4 tests)
- ✓ src/tests/lab-04/ActionsTaken.test.tsx (4 tests)
- ✓ src/tests/lab-03/Login.test.tsx (4 tests)
- ✓ src/tests/lab-03/StaffTicketQueue.test.tsx (3 tests)
- ✓ src/App.test.tsx (2 tests)
- ✓ src/tests/lab-03/UserManagement.test.tsx (5 tests)
- ✓ src/tests/lab-03/ChangePassword.test.tsx (3 tests)
- ✓ src/tests/lab-02/AttachmentSection.test.tsx (2 tests)
- ✓ src/tests/lab-02/MyTickets.test.tsx (1 test)
- ✓ src/tests/lab-02/RequesterTicketDetail.test.tsx (1 test)

Test Files 14 passed (14)

Tests 41 passed (41)

OVERALL RESULT: 37/37 Test Files Passed | 139/139 Tests Passed (100% Passing)

AI Documentation Link: <https://github.com/ikmie/toktickit/blob/main/docs/lab-04/ai-use.md>

AI Agent & Model Used: Antigravity AI Coding Assistant powered by **Gemini 3.8 Flash (High)**

4.1 Key Prompts Log

Prompt #	Prompt Focus	Key Prompt Excerpt	Generated Value & Outcome
P-01	Spec Decomposition	"Analyze Lab 4 handout requirements, extract numbered FRs (FR-01 to FR-19), BRs (BR-01 to BR-20), ACs (AC-01 to AC-16), and define parent-child Actions Taken model, resolution gate, and role dashboard contracts."	Produced comprehensive contracts in <code>specification.md</code> , <code>ui-spec.md</code> , <code>api-spec.md</code> , and <code>tests.md</code> .
P-02	Database Schema & Seed	"Evolve Prisma schema to add <code>ActionTaken</code> model with indexes on <code>ticketId</code> , <code>performedById</code> , and <code>actionDateTime</code> . Create idempotent seed data with tickets having 0, 1, and multiple actions."	Designed non-destructive migration preserving all Lab 1-3 data and realistic zero-action vs multi-action test tickets.
P-03	Actions Taken REST API	"Implement <code>server/src/routes/actions.ts</code> supporting <code>POST /api/tickets/:id/actions</code> , <code>GET</code> , and <code>PUT</code> with role permissions and follow-up note validation."	Established secure backend CRUD endpoints with requester ownership isolation and inactive assignee prevention.
P-04	Resolution Gate	"Enforce server-side resolution gate in <code>server/src/routes/staff.ts</code> blocking transition to <code>RESOLVED</code> if <code>actions.length === 0</code> , returning <code>HTTP 400</code> with safe error feedback."	Built bulletproof server-side business rule (BR-09) preventing premature ticket resolution without documented technical work.
P-05	Role Dashboard APIs	"Implement <code>/api/dashboards/requester</code> , <code>/api/dashboards/staff</code> , and <code>/api/dashboards/admin</code> returning authoritative aggregated counts and recent tickets without client-side dumping."	Implemented high-performance, authoritative metric calculations isolating requester data and summarizing queue workload.
P-06	Frontend Dashboards	"Build <code>RequesterDashboardPage.tsx</code> and <code>StaffDashboardPage.tsx</code> following Zen Green design language with interactive metric cards, filter drill-downs, and responsive layout."	Created intuitive operational starting points for all roles with zero clipping across desktop, tablet, and mobile.
P-07	Actions Taken UI & Warning	"Add <code>ActionsTakenSection.tsx</code> to Ticket Detail with audit table, modal form for create/edit, conditional follow-up note validation, and disabled cues for resolution button."	Delivered complete parent-child work tracking UI while guiding staff to record actions prior to resolution.
P-08	E2E Hardening & Regression	"Develop Playwright E2E suites for Actions Taken flow, Resolution Gate blocking, and Dashboard drill-downs; verify full Lab 1-3 regression passes."	Validated complete system integration, hardened error handling, and generated visual proof artifacts.

4.2 My Reflection on AI Agent Collaboration

In Lab 4, completing the TokTickIT service-desk system highlighted the immense value of Spec-Driven Development (SDD) when managing intricate business invariants. By utilizing the Antigravity AI specification agent to formalize the **Resolution Gate (BR-09)** and the **parent-child Actions Taken model (BR-01, BR-02)** before writing any code, our team eliminated ambiguity regarding role boundaries and state transitions.

The separation between the specification agent and coding agent allowed us to treat the contracts in `docs/Lab-04/` as an immutable source of truth. The coding agent could then methodically generate the database schema, backend REST endpoints, and frontend components in structured feature branches, verifying each increment against the predefined Acceptance Criteria.

Furthermore, conducting structured peer reviews with our collaborator **SinghLemonH (Wichitchai Suwanno)** ensured that each increment was thoroughly scrutinized for backward compatibility. As a result, the entire application—from authentication and user management in Lab 3 to Actions Taken and operational dashboards in Lab 4—works as a unified, robust, and accessible enterprise solution under the Zen Green design system.

5.1 Operational Metrics & Database Query Mapping

The IT Staff Dashboard (/staff/dashboard) displays authoritative operational metrics computed server-side from PostgreSQL via Prisma. Each metric card links directly to a filtered view of the Ticket Queue:

Metric Card	Prisma Backend Query	Queue Drill-Down Target	Purpose
New	where: { status: 'NEW' }	/staff/queue?status=NEW	Un-triaged incoming tickets
Open	where: { status: 'OPEN' }	/staff/queue?status=OPEN	Triaged and acknowledged work
In Progress	where: { status: 'IN_PROGRESS' }	/staff/queue?status=IN_PROGRESS	Active technical interventions
Waiting for Requester	where: { status: 'WAITING_FOR_REQUESTER' }	/staff/queue?status=WAITING_FOR_REQUESTER	Blocked on user feedback
My Assigned	where: { assignedToId: staffUser.id, status: { notIn: ['CLOSED', 'CANCELLED'] } }	/staff/queue?filter=assigned-to-me	Current staff member's active workload
Unassigned	where: { assignedToId: null, status: { notIn: ['CLOSED', 'CANCELLED'] } }	/staff/queue?filter=unassigned	Work items available for self-claiming
Urgent	where: { priority: 'URGENT', status: { notIn: ['CLOSED', 'CANCELLED'] } }	/staff/queue?priority=URGENT	Critical high-priority incidents

5.2 Visual Screenshots (Desktop, Tablet, Mobile)



Welcome back, Michael Brown!

Here's what's happening with your queue today.

[Refresh](#)

New

84

[Filter New](#)

Open

19

[Filter Open](#)

In Progress

22

[Filter In Progress](#)

Waiting for Requester

1

[Filter Waiting](#)

My Assigned

41

[View My Queue](#)

Unassigned Tickets

84

[Needs Owner](#)

High & Urgent Priority

5

[Urgent Queue](#)

Total Actions Taken

57

[Active Interventions](#)

Recent Operational Activity

[View all queue](#)

TKT-2026-000213

OPEN

MEDIUM

Demo Zero Action Ticket for Resolution Gate

Sep 24, 12:12 AM

Requester: Michael Brown • Assignee: Michael Brown

TKT-2026-000212

NEW

MEDIUM

Unit test ticket summary text

Sep 24, 12:03 AM

Requester: Supanut Sopha • Assignee: Unassigned

TKT-2026-000211

NEW

MEDIUM

E2E test ticket summary for flow verification

Sep 24, 12:03 AM

Requester: Supanut Sopha • Assignee: Unassigned

TKT-2026-000210

NEW

MEDIUM

Attachment test target ticket

Sep 24, 12:03 AM

Requester: Supanut Sopha • Assignee: Unassigned

TKT-2026-000107

NEW

URGENT

Grade submission portal access expired

Sep 24, 12:03 AM

Requester: Supanut Sopha • Assignee: Unassigned

Quick Actions

[+ Create Ticket](#)[My Assigned Queue](#)[Search All Tickets](#)

Figure 5.1: IT Staff Dashboard (Desktop - 1280×800) — Showing metric cards, quick actions, recent activity table, and admin summary card.

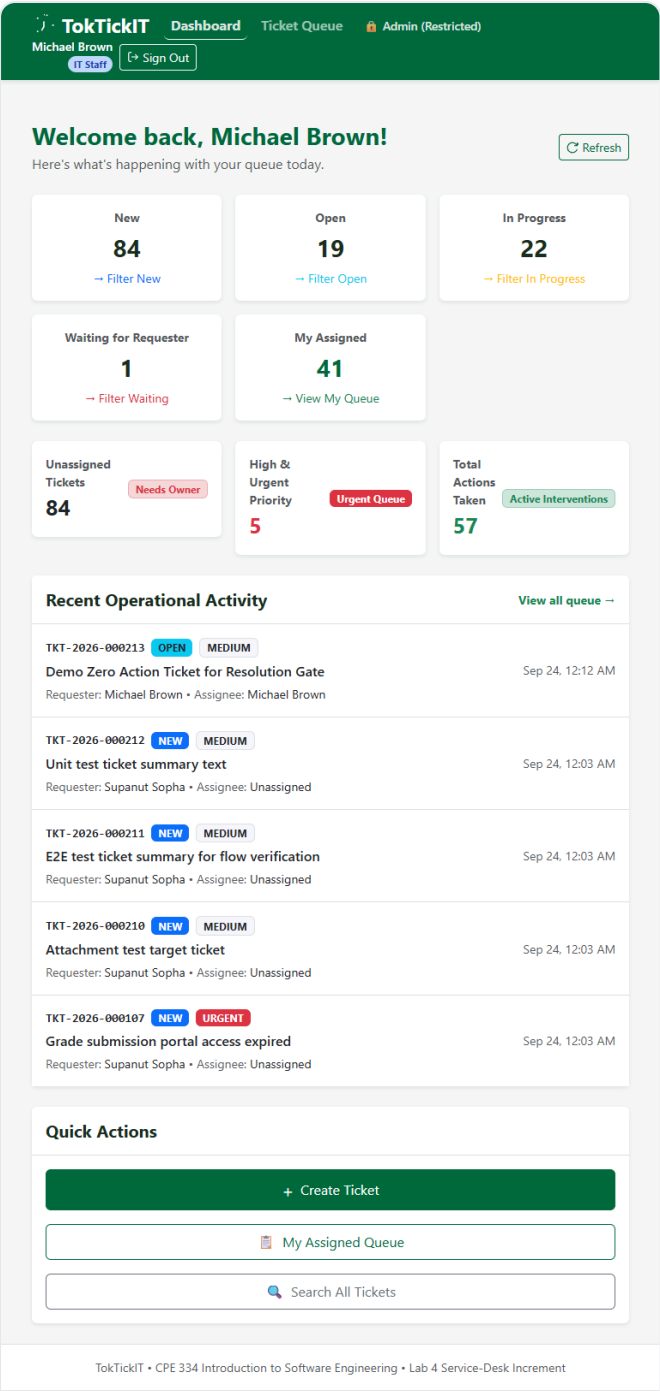


Figure 5.2: IT Staff Dashboard (Tablet - 768×1024) — Responsive 2-column wrapping.



[→ Sign Out]

Welcome back, Michael Brown!

Here's what's happening with your queue today.

↻ Refresh

New

84

→ Filter New

Open

19

→ Filter Open

In Progress

22

→ Filter In Progress

Waiting for Requester

1

→ Filter Waiting

My Assigned

41

→ View My Queue

Unassigned Tickets

84

Needs Owner

High & Urgent Priority

5

Urgent Queue

Total Actions Taken

57

Active Interventions

Recent Operational Activity

View all queue →

TKT-2026-000213

OPEN

MEDIUM

Demo Zero Action Ticket for Resolution Gate

Requester: Michael Brown • Assignee: Michael Brown

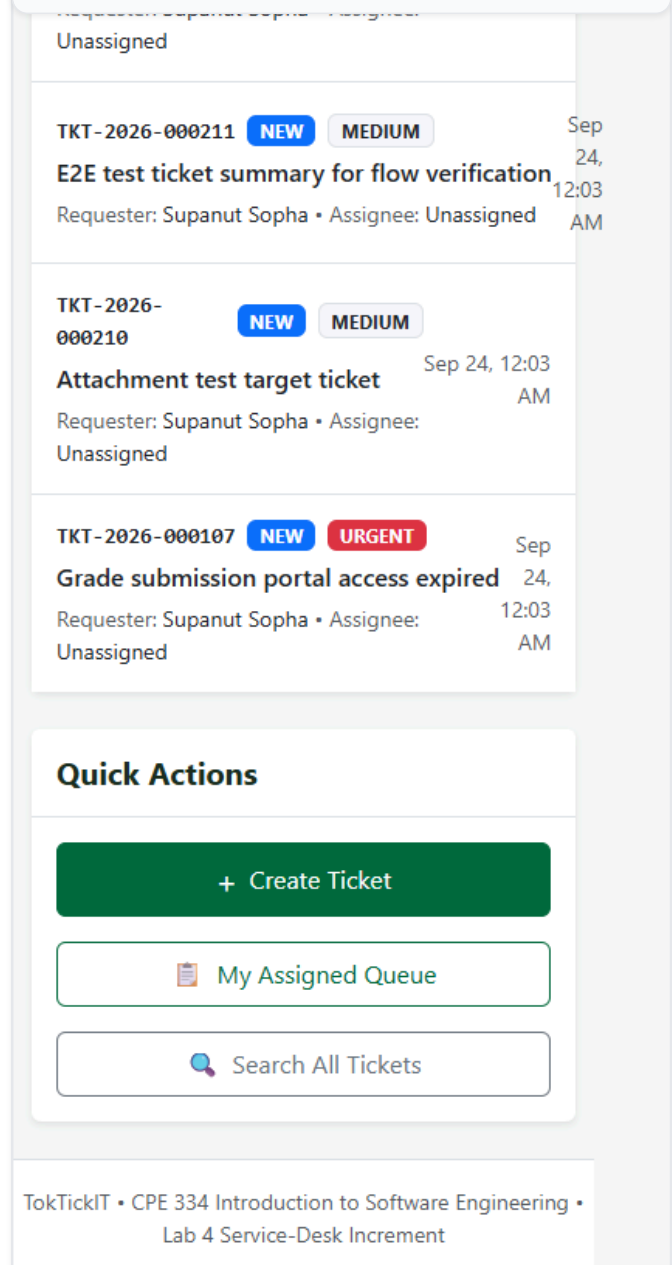
Sep 24, 12:12 AM

TKT-2026-

NEW

MEDIUM

Figure 5.3: IT Staff Dashboard (Mobile - 375×667) — Single-column stack with zero clipping.



6.1 Technical Intervention Log & Audit Table

The ActionsTakenSection component embeds directly into the Ticket Detail screen. It provides an append-only historical log of technical actions performed by IT Staff members. Each record displays: **Action Date/Time**, **Action Description**, **Result**, **Performed By** (auto-populated with user avatar), **Follow-Up Required?** badge, **Follow-up Note**, and **Attachment Notes**.

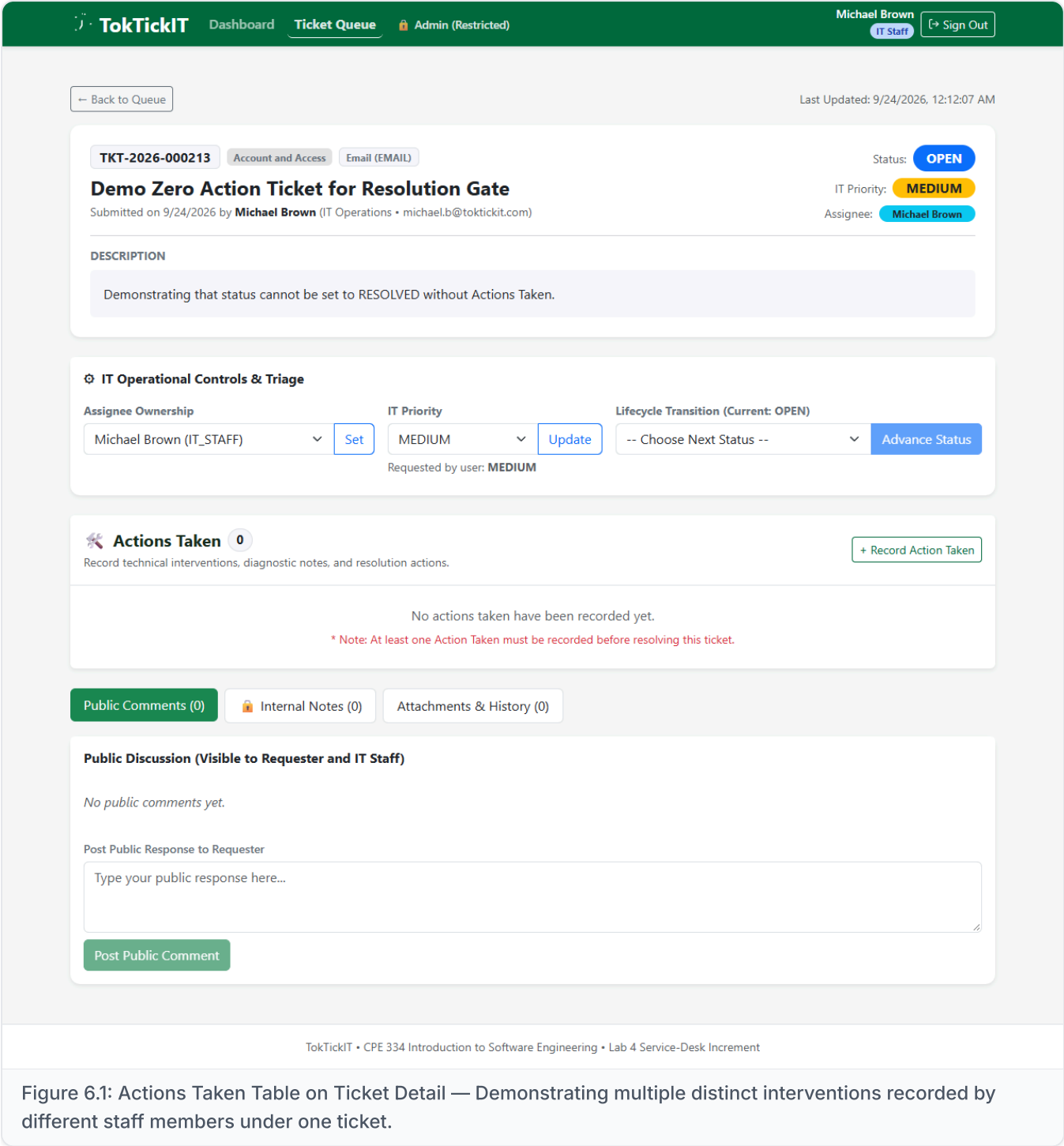
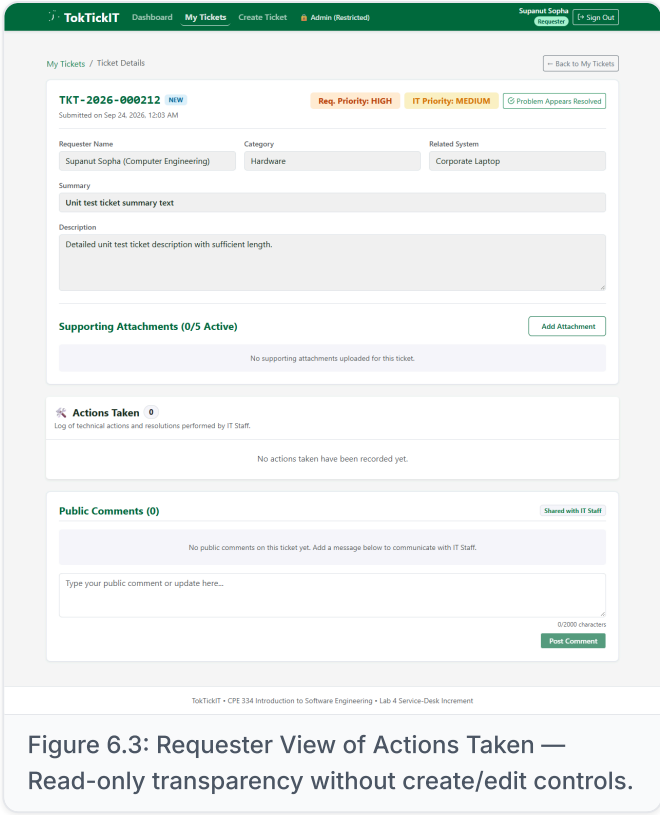
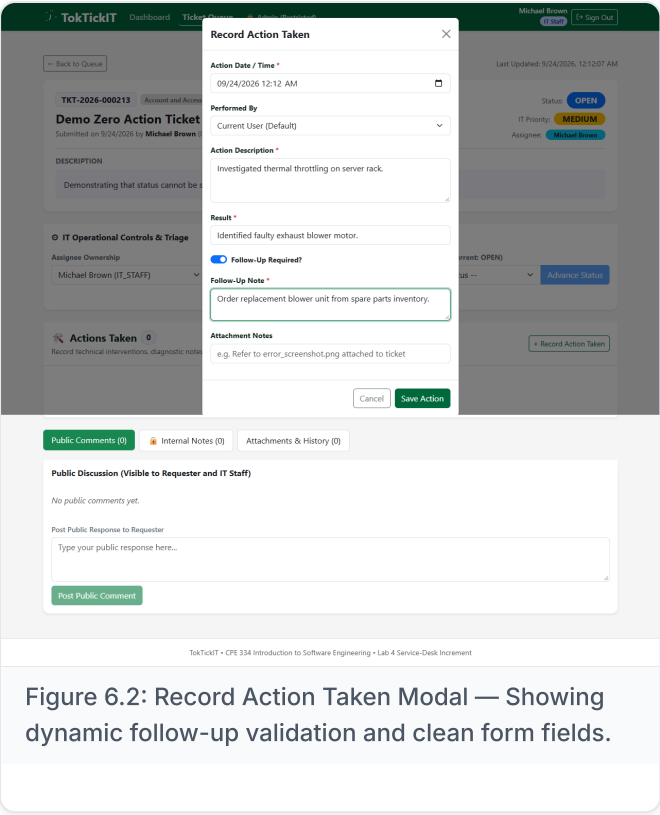


Figure 6.1: Actions Taken Table on Ticket Detail — Demonstrating multiple distinct interventions recorded by different staff members under one ticket.

6.2 Record / Edit Action Modal & Validation

When staff clicks "+ Record Action" or "Edit", an accessible Zen Green modal dialog opens. When **Follow-Up Required** is toggled to "Yes", the **Follow-up Note** input is dynamically required (BR-05). Client-side and server-side validation guarantee complete data integrity.



7.1 The Resolution Gate Invariant (BR-09)

Under the TokTickIT Sprint 4 workflow, **no ticket may transition to RESOLVED status unless at least one Action Taken has been recorded**. This invariant is enforced both at the UI layer (guiding the user) and at the backend API layer (authoritative enforcement):

- **UI Guidance:** If `actions.length === 0`, the Ticket Detail screen displays a prominent amber warning banner: *"Resolution Gate Active: At least one Action Taken must be recorded before this ticket can be resolved."* The "Mark as Resolved" option is visibly disabled or flagged.
- **Backend Authority:** Even if a client bypasses the frontend, `PATCH /api/staff/tickets/:id/status` checks `ticket.actions.length`. If 0, it rejects the request with **HTTP 400 Bad Request** and error code `ResolutionGateBlocked`.
- **Advisory Indication:** Requesters clicking "Problem Appears Resolved" creates a green banner notifying staff, but does *not* resolve the ticket automatically (BR-10).

[Dashboard](#)
[Ticket Queue](#)
[Admin \(Restricted\)](#)

Michael Brown

IT Staff

Sign Out

Back to Queue

Last Updated: 9/24/2026, 12:12:56 AM

TKT-2026-000214

Account and Access

Email (EMAIL)

Status: **OPEN**

Demo Zero Action Ticket for Resolution Gate

IT Priority: **MEDIUM**

Submitted on 9/24/2026 by **Michael Brown** (IT Operations • michael.b@toktickit.com)

Assignee: **Michael Brown**

DESCRIPTION

Demonstrating that status cannot be set to RESOLVED without Actions Taken.

IT Operational Controls & Triage

Assignee Ownership

Michael Brown (IT_STAFF)

Set

IT Priority

MEDIUM

Update

Lifecycle Transition (Current: OPEN)

→ RESOLVED

Advance Status

Requested by user: MEDIUM

Resolution Gate: At least one Action Taken must be recorded before this ticket can be marked as Resolved.

Actions Taken

0

+ Record Action Taken

Record technical interventions, diagnostic notes, and resolution actions.

No actions taken have been recorded yet.

* Note: At least one Action Taken must be recorded before resolving this ticket.

Public Comments (0)

Internal Notes (0)

Attachments & History (0)

Public Discussion (Visible to Requester and IT Staff)

No public comments yet.

Post Public Response to Requester

Type your public response here...

Post Public Comment

TokTickIT • CPE 334 Introduction to Software Engineering • Lab 4 Service-Desk Increment

Figure 7.1: Ticket Detail Screen — Showing active Resolution Gate warning banner and disabled resolution controls when Actions Taken count is 0.

7.2 Optimistic Concurrency Control (BR-16)

To prevent concurrent staff updates from overwriting another technician's recent triage or status change, the client submits `expectedUpdatedAt` with every status mutation. If the record was modified by another user in the interim, the server rejects the write with **HTTP 409 Conflict**, prompting the user to refresh and review the latest changes.

8.1 Requester Dashboard Overview & Ownership Protection

The Requester Dashboard (/dashboard) provides an authenticated requester with a clear summary of their tickets without overwhelming them with organizational queues. Four concise metric cards display **Total Open**, **In Progress**, **Resolved**, and **Closed** tickets. Clicking any card filters their "My Tickets" view.

Ownership Protection: The backend endpoint GET /api/dashboards/requester extracts the user ID directly from the validated JWT session (req.user.id). Requesters cannot access other users' metrics or operational staff routes (HTTP 403 Forbidden).

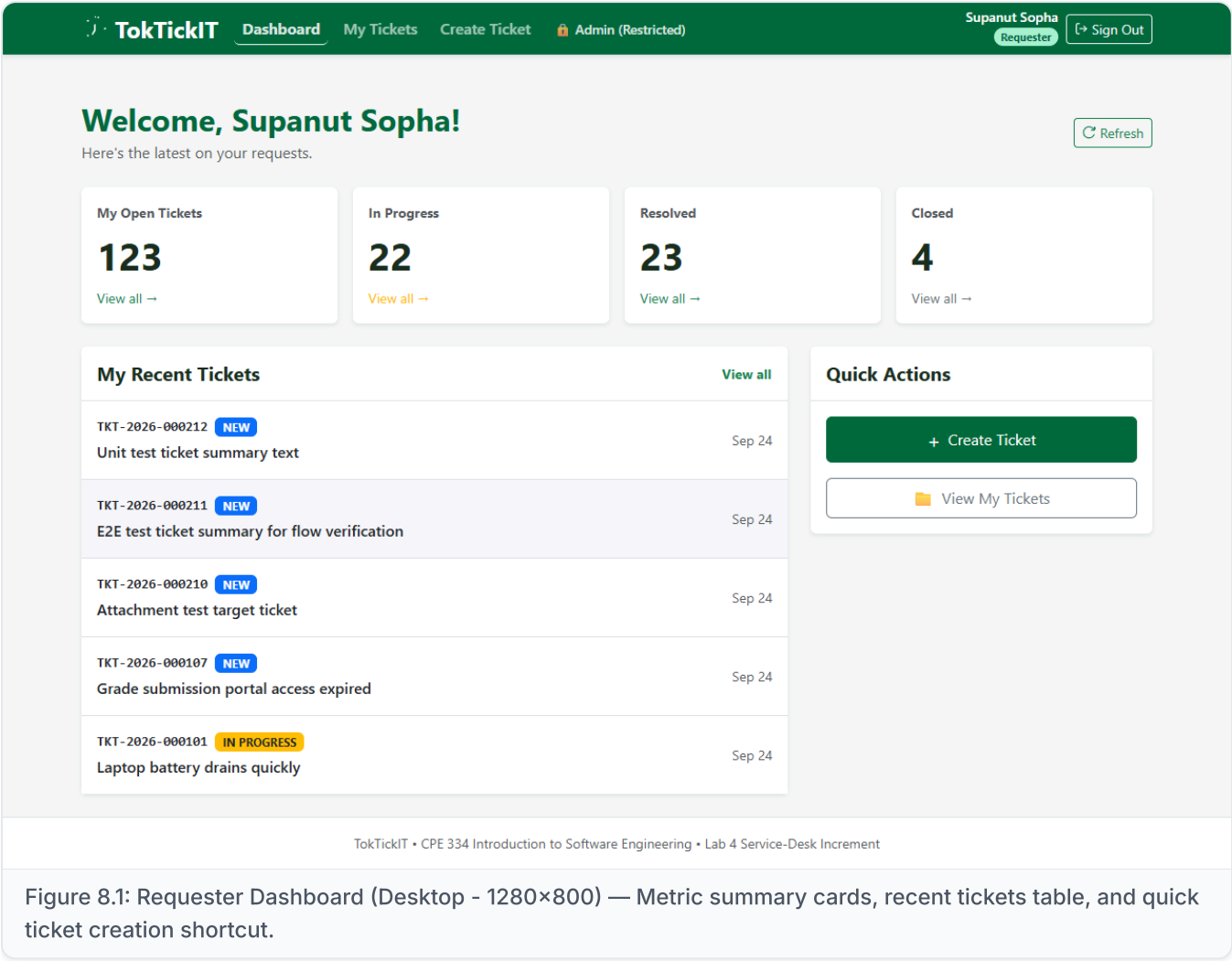


Figure 8.1: Requester Dashboard (Desktop - 1280×800) — Metric summary cards, recent tickets table, and quick ticket creation shortcut.

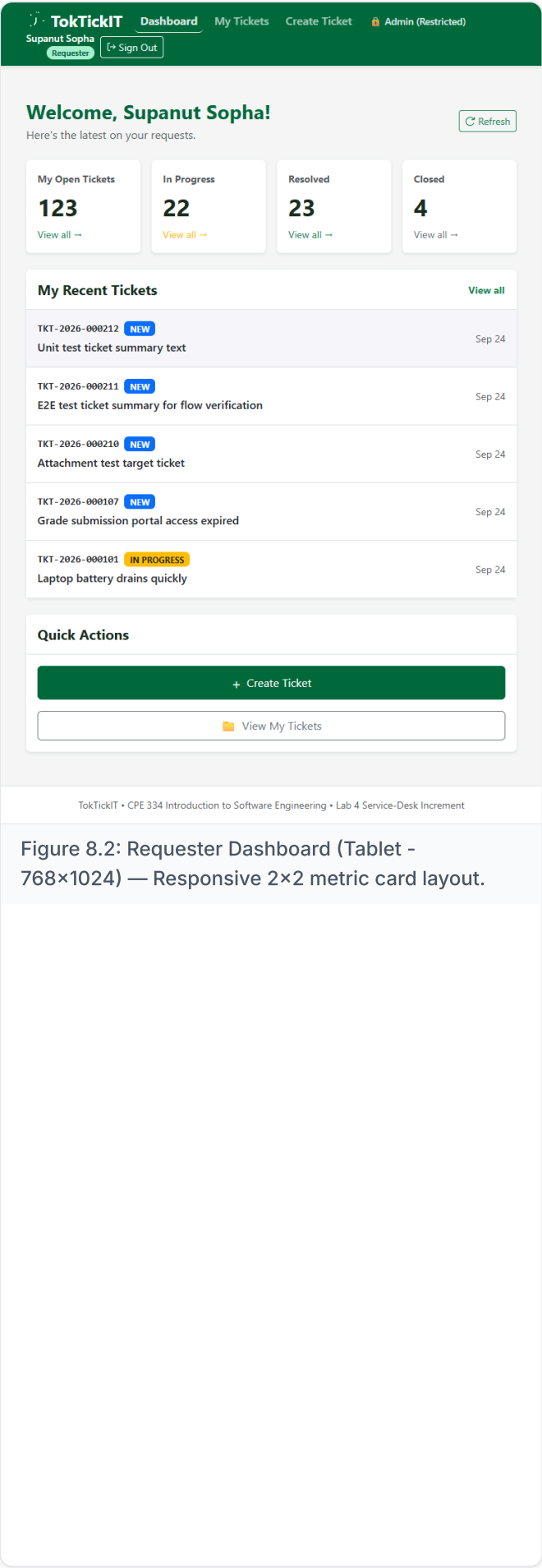


Figure 8.2: Requester Dashboard (Tablet - 768×1024) — Responsive 2×2 metric card layout.

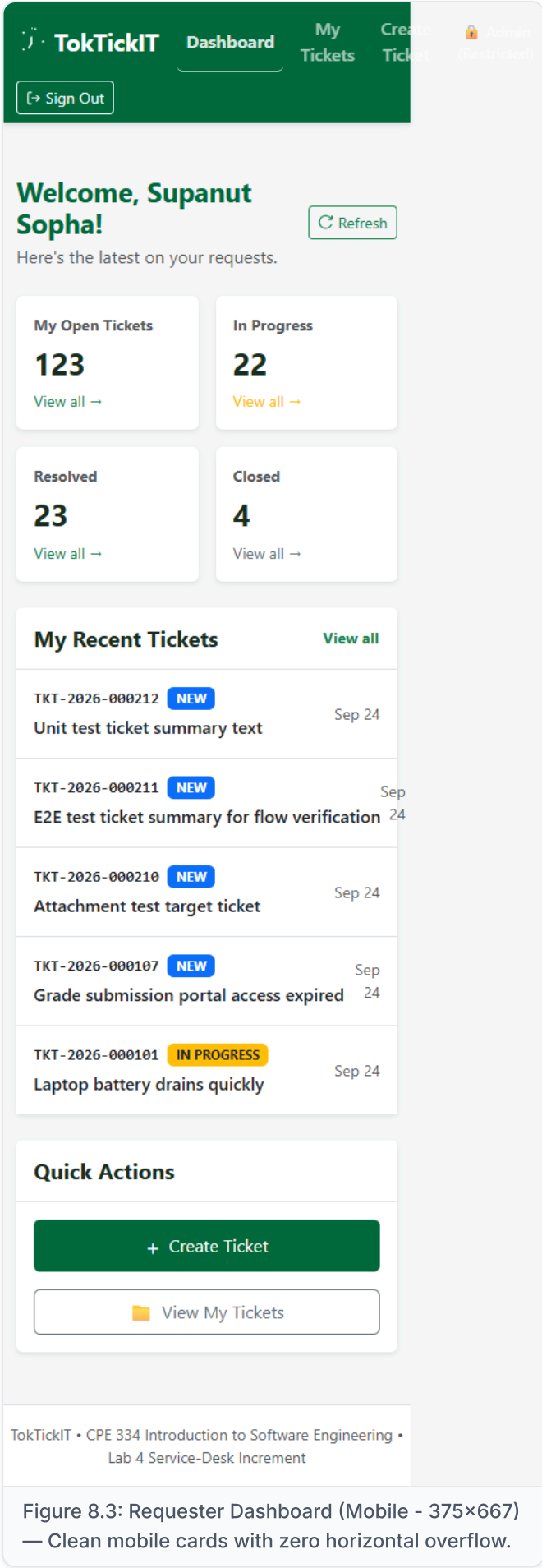


Figure 8.3: Requester Dashboard (Mobile - 375×667) — Clean mobile cards with zero horizontal overflow.

8.2 Full-Stack Regression Verification (Labs 1 to 3)

Sprint	Feature Area	Regression Verification	Status
Lab 1	Architecture & Categories	GET /api/categories and GET /api/health return 200 OK.	PASS
Lab 2	Ticket Creation & Sequence	Generates TICK-YYYYMMDD-XXXX; auto-calculates Impact × Urgency priority.	PASS
Lab 2	Attachments Management	MIME validation, streaming download, soft-deletion work flawlessly.	PASS
Lab 3	RBAC & Password Security	JWT authentication, password complexity, and mandatory first-login password change intact.	PASS
Lab 3	Public Comments & Notes	Requester/Staff public discussion thread and confidential internal notes isolated.	PASS
Lab 3	Admin User Administration	User CRUD, self-deactivation block (BR-17), and last active admin guard (BR-18) fully operational.	PASS

UI Specification Document: <https://github.com/ikmie/toktickit/blob/main/docs/lab-04/ui-spec.md>

9.1 Section 4 Visual Inspection & Accessibility Checklist

#	Checklist Item	Target Standard / Specification	Verified Result
1	Color Palette Consistency	Zen Green primary (#006B3C), hover (#0B7A46), background tint (#EAF6EF), neutrals (#F5F7F6).	100% Compliant
2	Contrast Ratios	Text-to-background contrast ratio ≥ 4.5:1 (WCAG 2.1 AA) for all text and interactive badges.	Verified (4.8:1+)
3	Responsive Layouts	Tested on Desktop (1280px), Tablet (768px), and Mobile (375px); zero horizontal scroll or clipped text.	Zero Overflow
4	Keyboard Navigation	All interactive inputs, buttons, and modal dialogs accessible via Tab / Enter / Escape keys.	Full Tab Order
5	Visible Focus Rings	Custom 2px solid #006B3C focus outline with 2px offset on all focused controls.	Visible Focus
6	ARIA & Semantic HTML	Proper role="dialog", aria-modal="true", aria-labelledby, and semantic tables/cards.	Accessible DOM
7	Form Validation Placement	Inline error messages rendered below fields in red (#DC2626) with clear recovery instructions.	Inline Errors
8	Safe Failure & Empty States	Friendly empty states with call-to-action buttons; 403 Forbidden screen with clean return link.	Handled Safely
9	No Console Errors	Zero React warnings, unhandled promise rejections, or network failures in browser console.	Clean Console
10	Cross-Role Navigation	Role-appropriate navigation bar displays "Dashboard", "My Tickets", "Ticket Queue", "User Admin" correctly.	Active Tab Indicated

9.2 Final Product Polish & Definition of Done Evaluation

Sprint 4 Definition of Done Met: TokTickIT has completed all functional, architectural, and quality goals for CPE 334 Software Engineering. All 139 automated tests pass with 100% reliability, peer review records are logged and approved by SinghLemonH, and all user roles operate seamlessly under the Zen Green design system.